Reinforcement learning I

Arturo Gil y Juan José Cabrera

27 de octubre de 2026

1. Introducción

La mayor parte de los conceptos vistos hasta el momento en la asignatura se pueden clasificar como algoritmos de aprendizaje supervisado. Dentro de este tipo de aprendizaje consideramos algoritmos a los que se le proporciona un conjunto de datos de entrenamiento etiquetados. El rol del algoritmo es encontrar una función que permita hallar una mapeo lo más aproximado posible entre las entradas y las salidas proporcionadas. En el ámbito del aprendizaje por refuerzo (*reinforcement learning*) se plantea un problema que difiere significativamente del anterior: se desea que un agente alcance algún grado de aprendizaje o inteligencia sin utilizar datos de entrenamiento etiquetados. Los únicos datos disponibles son aquellos que genera el agente mientras interactúa con el entorno. A continuación indicamos algunos ejemplos de aprendizaje con refuerzo (*reinforcement learning*):

- Un robot en un laberinto (figura 1): asuma que tenemos un robot en un laberinto. Se trata de un robot omnidireccional. Así pues, en cada momento el robot puede moverse en 4 direcciones (izquierda, derecha, arriba o abajo). Consideramos que en todo momento podemos conocer la posición del robot en el laberinto. El objetivo del problema es que el robot sea capaz de encontrar el camino óptimo que le lleve a la salida, cuando el robot parte desde diferentes puntos de inicio. Así pues, el aprendizaje consiste en que el robot se forme una especie de mapa mental que le permita tomar la mejor acción (moverse a la izquierda, derecha, arriba abajo) en cada punto del mapa y que estas acciones lo lleven siempre a la salida.

- Imaginemos ahora que tenemos un robot bípedo como el mostrado en la figura 2a. En este caso, nos gustaría poder programar al robot para que pudiera andar. Imaginemos también que tenemos un modelo dinámico que nos permite simular el sistema en el ordenador (figura 2b). Finalmente, sabemos que la marcha bípeda es extremadamente compleja, pues todas las variables del sistema están acopladas. Igualmente, las ecuaciones dinámicas del sistema son altamente no lineales y están acopladas: cuando se aplica un par a una articulación (mediante un motor) también se generan reacciones de fuerzas y pares en otras articulaciones. En este caso, en vez de plantear un control de las variables articulares, ¿por qué no utilizamos un algoritmo de aprendizaje que use el simulador y pruebe diferentes soluciones para que el agente pueda caminar? Durante la simulación, cada vez que el robot realice un movimiento que le permita avanzar, recibirá un premio. Por contra, cada vez que el robot se caiga al suelo, se castigará.

Merece la pena decir aquí que, como en casi todos los ámbitos del conocimiento, existen soluciones alternativas. Así pues, en el primero de los problemas planteados, un ingeniero especialista en el ámbito de la robótica móvil nos diría que también se podría plantear la construcción de un mapa de ocupación con alguna técnica de *mapping* o *SLAM* para después utilizar el algoritmo A^* y planificar una ruta óptima (este algoritmo se ha estudiado en una práctica anterior). En el caso de los robots caminantes, también hay otras formas de plantear el problema: típicamente, un ingeniero definiría con cuidado el *gait* de cada articulación y después intentaría optimizarlo en base a los parámetros dinámicos del robot.

2. Objetivos

En esta práctica se recordarán algunos conceptos de aprendizaje por refuerzo y, en concreto, del algoritmo conocido como Q-learning tabular. Seguidamente, se verán algunos ejemplos de Q-learning

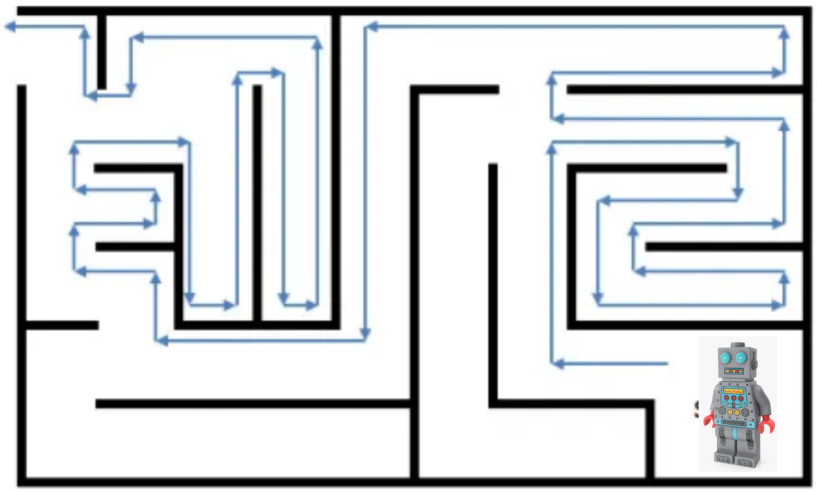
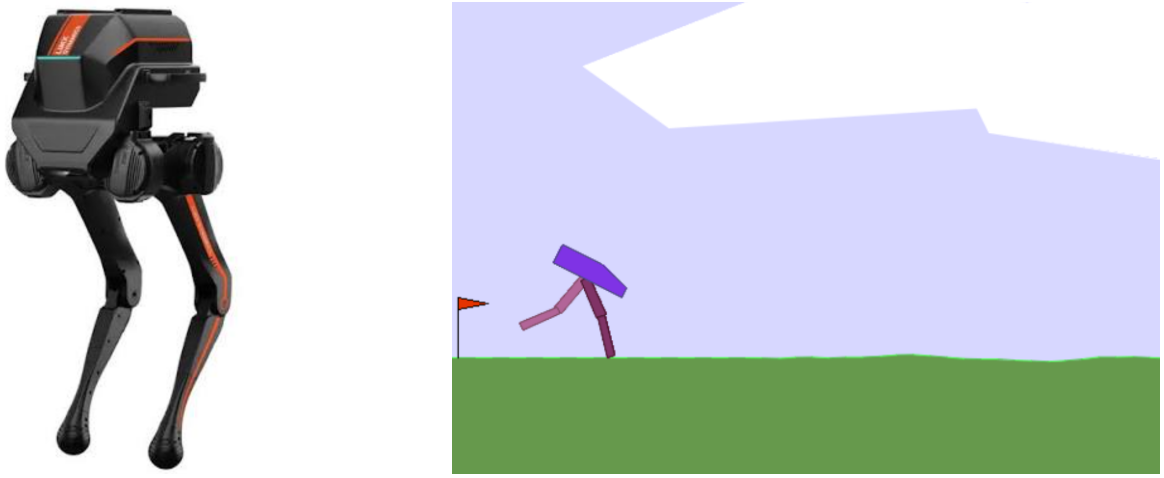


Figura 1: Un robot en un laberinto



(a) Un robot bípedo.

(b) Un robot bípedo simulado en Gymnasium.

Figura 2: Un robot bípedo real (arriba) y simulado (abajo).

usando la librería Gymnasium para python. **El estudiante experimentará de diferentes formas con estos entornos y observará qué ocurre con el aprendizaje cuando se cambian los hiperparámetros del algoritmo.**

3. El repositorio de prácticas y Gymnasium

Se os proporciona un repositorio de Github con algunos scripts que usan el algoritmo Q-learning y están basados en la librería Gymnasium (https://gymnasium.farama.org/). Se trata de una librería para python que incluye una serie de entornos y agentes y es especialmente adecuada para probar y desarrollar algoritmos de aprendizaje por refuerzo. Si quieres observar estos entornos, visita https://gymnasium.farama.org/environments/classic_control/, por ejemplo para ver los ejemplos de reinforcement learning para problemas clásicos de control (el péndulo invertido, cart-pole, etc...). La librería es realmente interesante y permite, entre otras cosas, entrenar a un agente para que sea capaz de jugar a cualquiera de los 106 juegos de la consola Atari 2600 (comercializada en 1977).

4. Algunos conceptos de *reinforcement learning*

4.1. Q-learning

El algoritmo Q-learning es uno de los más famosos en el ámbito del aprendizaje por refuerzo. Se trata un algoritmo que no precisa de modelos del entorno ni del agente. El algoritmo busca la estimación de una tabla Q donde cada fila representa un estado s y las columnas representan las posibles acciones a que puede realizar el agente en cada estado. En cada celda de la tabla se almacena un valor $Q(s, a)$ que estima la recompensa futura (sobre todos los estados futuros hasta el final del episodio) de tomar la acción a cuando el agente se encuentra en el estado s. La tabla se calcula durante el entrenamiento usando la ecuación de Bellman. Durante la fase de funcionamiento (test), el agente ejecuta la acción a para la cual el valor en la tabla es máximo. También consideramos que en un momento determinado de tiempo t, el estado del agente y del entorno se puede describir mediante $s_t \in S$ (es decir, un estado s_t dentro del conjunto de posibles estados que describen al agente y al entorno). Asumimos que en ese mismo instante el agente decide realizar una acción $a_t \in A$. Contamos con un modelo de transición de estados (también llamado un modelo de movimiento). Este modelo se representa de dos formas diferentes:

- En el caso más sencillo: $s_{t+1} = \delta(s_t, a_t)$. Dado que el robot se encuentra en s_t, si se ejecuta la acción a_t entonces el estado cambiará a ser s_{t+1}. En el ejemplo inicial del laberinto, el robot se moverá a la siguiente posición, dependiendo de la acción a_t que se realice (arriba, abajo, izquierda o derecha) y, además, dependiendo de si hay una pared en el laberinto o si el espacio está libre. Esta situación contempla el caso en el que el modelo de movimiento del agente es determinista.
- En el caso más general, hablaremos de un modelo de transición de estado probabilístico. Estos modelos son instancias de MDP (*Markov Decision Processes*) y se representan mediante una función de probabilidad condicionada $P(s_{t+1}|s_t, a_t)$. Es decir, dado que el agente y entorno se encuentran en s_t y se ejecuta la acción a_t, podemos establecer una función de probabilidad sobre el estado futuro s_{t+1}.

4.2. Estados y acciones

En esta práctica trabajaremos con el caso en el que el estado y las acciones se pueden describir de forma discreta. En una práctica posterior (*reinforcement learning II*) trabajaremos con el caso en el que el estado es continuo y las acciones discretas. El caso más complejo es aquél en el que tanto el estado como las acciones son continuas. Ejemplos:

- Estado y acciones discretas: *Taxi* y *Cliff Walking*.
- Estado continuo y acciones discretas: *Lunar Lander*.
- Estado y acciones continuas: *Bipedal Walker*.

4.3. Etapas y episodios

Una etapa (paso o step) del algoritmo comprende el caso en el que el agente se encuentra en un estado s_t, realiza una acción a_t que la conduce a otro estado s_{t+1} (y recibe una recompensa r_t).

Un episodio incluye un número variable de pasos. En un episodio existe un inicio y un final: generalmente el agente debe ser capaz de cumplir una misión. En algunos procesos podemos establecer de forma natural cuándo se inicia y se finaliza la misión. En el caso del robot en el laberinto, el episodio comienza con el robot en la casilla inicial y finaliza cuando el robot llega a la casilla de salida (o bien cuando se ha superado un número máximo de pasos). Realizada la última acción por el robot, la misión finaliza y, por tanto, podemos decir que el episodio acaba.

En otras situaciones, como en el caso en el que se quiere controlar un proceso continuo, el concepto de episodio no es adecuado, pues el control debe funcionar *ad eternum*. Será importante que el estado inicial s_0 varíe en el comienzo de cada episodio. Este hecho es fundamental para que el agente aprenda de una forma general.

4.4. Entrenamiento

El objetivo del entrenamiento es maximizar la suma de las recompensas de cada episodio. Esto supone que las recompensas deben estar bien definidas. Según se definan las recompensas, así será el resultado final del aprendizaje. Así pues, los pesos de las recompensas deben ser elegidos cuidadosamente y con rigor para que se produzca un correcto aprendizaje. En los casos de aprendizaje en los que se desea que un robot camine, se premia con +1 cada vez que el robot avanza en la dirección deseada y con -10, por ejemplo, cuando el robot pierde el equilibrio y se cae.

4.5. Train y test

En el paradigma del aprendizaje con refuerzo, se plantean generalmente dos fases separadas:

- Entrenamiento (*train*): se plantea inicialmente una etapa en la que el agente se entrena, obteniendo conocimiento sobre el entorno, según realiza acciones y obtiene recompensas.
- Prueba, funcionamiento o *test*: se plantea, después, otra etapa en la que se prueba el algoritmo con el objetivo de medir sus capacidades.

En ambas etapas nos interesa medir el comportamiento del agente para evaluar sus conocimientos. También es frecuente que tengamos que variar los parámetros del algoritmo para mejorar su aprendizaje. Aquí se apunta alguna información que debe ser almacenada en cada episodio para observar el aprendizaje:

- Suma de recompensas en cada episodio.
- Histograma de los estados visitados en todos los episodios. Para comprobar si el agente ha podido visitar todos los estados posibles y, en cada uno de ellos, ha podido probar todas las acciones disponibles.
- Imagen de la tabla Q. Permite visualizar rápidamente las acciones para diferentes estados discretos.

4.6. Exploitation, exploration and reward

Definimos a continuación los conceptos de explotación, exploración y recompensa (*exploitation, exploration and reward*):

- *Exploitation*: hace referencia al cálculo de una acción a realizar cuando se hace uso del conocimiento actual sobre la acción que produce una mejor recompensa futura. En el algoritmo Q-learning, el conocimiento se almacena en la tabla Q.
- *Exploration*: se habla de exploración cuando el agente toma decisiones aleatorias. Al tomar acciones aleatorias, el agente puede llegar a estados que permiten mejorar el conocimiento que tenemos sobre las recompensas y el entorno. El concepto de exploración responde a la pregunta ¿qué pasa si estando en este estado llevo a cabo una acción aleatoria a_t, obtendré una recompensa futura mayor?

En general, explotar el conocimiento que tenemos permite maximizar la recompensa actual, pero la exploración permite probar nuevas formas de resolver el problema que podrían producir una recompensa total futura mayor.

En un único paso del algoritmo no es posible explorar y explotar al mismo tiempo, pues únicamente podemos aplicar la acción una vez y, a continuación, el agente pasará a estar en un estado diferente. Así pues, en ocasiones se habla del conflicto entre la exploración y la explotación del conocimiento que se tiene en un determinado momento: el agente se podría plantear en cada momento, ¿qué hago?, ¿explorar o explotar el conocimiento que he recabado hasta ahora? Así pues, en un momento determinado del ciclo de aprendizaje, nos podríamos preguntar, ¿cómo de seguros estamos de que tenemos un conocimiento correcto sobre la tabla $Q(s, a)$. Por ejemplo, imaginemos que el agente ha llegado a un estado $s_t = 8$ y consulta la tabla $Q(s, a)$ donde se obtiene una acción a_t=GO RIGHT. Después de realizar esa acción, el agente obtiene una recompensa $r_t = 1$. Imaginemos que ahora estamos en un episodio diferente y hemos llegado al mismo estado $s_t = 8$ de nuevo. Nos preguntamos ahora si a) debemos repetir la acción

anterior y asegurar una recompensa inmediata de 1 punto o, b) elegir otra acción diferente, esperando que esto dé lugar, al final del episodio, una recompensa final mayor.

La forma más académica de resolver el dilema entre la exploración y la explotación es plantear qué incertidumbre tenemos sobre la matriz Q. Imaginemos que el agente que está tomando decisiones es un estadístico. Podría evaluar las acciones que se habían tomado con anterioridad y calcular la incertidumbre con la que se conoce esta información. Cuando la incertidumbre sobre las estimaciones sea baja, el estadístico debería decidir que ya no es necesario más acciones exploratorias y se debe usar únicamente las acciones que indique la tabla Q.

4.7. Algoritmo greedy y epsilon greedy

En esta práctica se plantea una forma sencilla de articular la exploración y la explotación. Esta forma de proceder se denomina ϵ-*greedy* (epsilon *greedy*). Esta política funciona de la siguiente manera: se inicializa $\epsilon = 1$ y se programa una función que decrece con el paso de los episodios de la siguiente manera:

```
for episode in episodes:
    environment.reset()
    epsilon = 1.0
    while True
        # running episode i
        p=normrnd()
        if p < epsilon:
            exploration: random action
        else
            exploitation: computed action
        s_{t+1} = environment.step(action)
    epsilon = delta*epsilon
```

Así pues, con probabilidad ϵ se explora y con probabilidad $1 - \epsilon$ se explota el conocimiento. El valor de ϵ se va reduciendo en cada episodio, pues es preferible tomar acciones aleatorias durante los primeros episodios del aprendizaje y refinar el conocimiento durante los episodios finales. Esta estrategia se denomina ϵ-greedy con decaimiento. La estrategia consiste en reducir ϵ según avanza la simulación al multiplicarlo por un factor δ:

$$\epsilon_{t+1} = \epsilon_t \cdot \delta \tag{1}$$

Esta operación se hace, bien durante cada paso de la simulación o bien en cada episodio. En esta práctica, reduciremos ϵ en cada episodio. Además, se fija un valor mínimo ϵ_{min}:

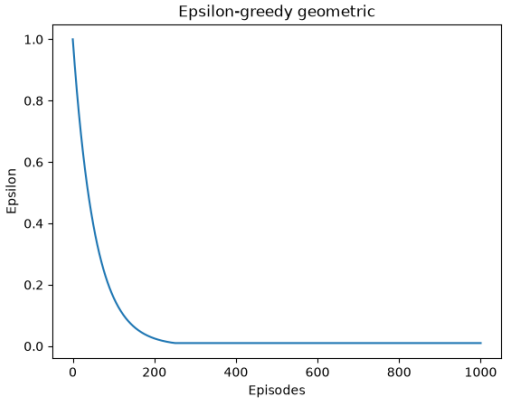
$$\epsilon_{t+1} = \text{máx}(\epsilon_{min}, \epsilon_t \cdot \delta) \tag{2}$$

Generalmente, se calcula δ para que ϵ vaya de su valor máximo a su valor mínimo en una fracción de los episodios de entrenamiento. Es típico calcular el factor de decaimiento δ para que pase de ϵ_{max} a ϵ_{min} en un 25 % del total de episodios de entrenamiento. Así, si por ejemplo, el total de episodios de entrenamiento es $N = 1000$, entonces $N_{target} = N \cdot 0,25 = 250$ y calculamos:

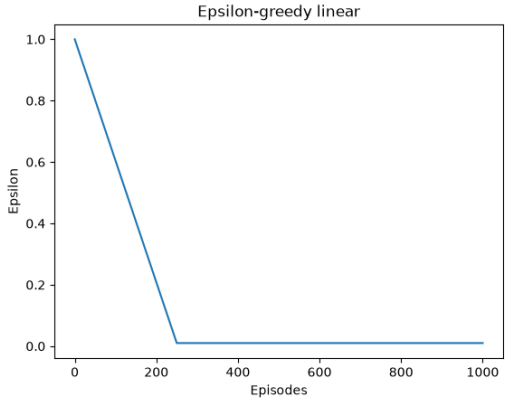
$$\delta = \left(\frac{\epsilon_{\text{min}}}{\epsilon_{\text{max}}}\right)^{\frac{1}{N_{target}}}$$

Por ejemplo:

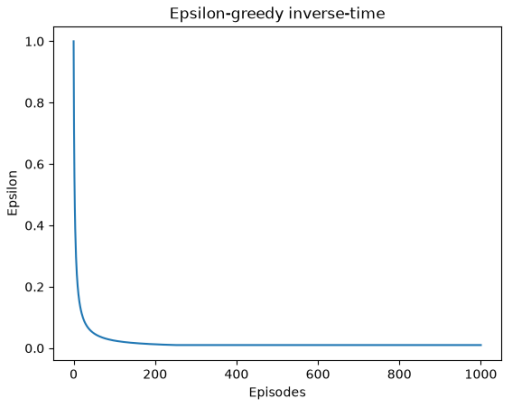
- $N = 1000$.
- $N_{target} = 250$.
- $\epsilon_{max} = 1.0$ (valor inicial).
- $\epsilon_{min} = 0.01$ (valor final).
- $\delta = 0.98174$.



(a) `EpsilonGreedyGeom`.



(b) `EpsilonGreedyLinear`.



(c) `EpsilonGreedyInverseTime`.

Figura 3: Valores de ϵ por episodio para las diferentes clases `EpsilonGreedyGeom`, `EpsilonGreedyLinear` y `EpsilonGreedyInverseTime`.

Es fácil comprobar que $\epsilon_{max} \cdot \delta^{250} \approx 0.01$. Se usan diferentes tipos de funciones para variar el valor de ϵ en función de los episodios. Se listan las más importantes:

- Geométrica: $\epsilon_{t+1} = \text{máx}(\epsilon_{\text{mín}}, \epsilon_t \cdot \delta)$, con $0 < \delta < 1$.
- Exponencial:
- Lineal: $\epsilon_t = \text{máx}(\epsilon_{\text{mín}}, \epsilon_0 - \delta \cdot t)$, siendo δ es un paso fijo.
- Tiempo inverso: $\epsilon_t = \frac{\epsilon_0}{1+\delta \cdot t}$

Se proporciona al estudiante una clase base `EpsilonGreedy` y tres clases derivadas (`EpsilonGreedyGeom`, `EpsilonGreedyLinear` y `EpsilonGreedyInverseTime`) que implementan cada uno de los anteriores protocolos para ϵ. En la figura X.X se muestra el resultado de ejecutar los objetos de las clases derivadas (`EpsilonGreedyGeom`, `EpsilonGreedyLinear` y `EpsilonGreedyInverseTime`) con $N = 1000$, $N_{target} = 250$, $\epsilon_{max} = 1.0$ y $\epsilon_{min} = 0.01$.

NOTA: En los problemas que resolveremos en estas prácticas usaremos la regla del 25/75 (frecuente en el área del aprendizaje con refuerzo [SB18]). Así, si pensamos entrenar durante 1000 episodios, usaremos 250 a explorar (con alta probabilidad) y 750 para refinar la solución. Estos porcentajes, claro está, se deben variar para adaptarse al problema en cuestión: de esta adaptación dependerá en gran medida el resultado del entrenamiento.

4.8. Estimación de la tabla Q

En el algoritmo Q-Learning, el valor de un elemento de la tabla $Q(s_t, a_t)$ representa una estimación de todas las recompensas futuras que se obtendrían hasta el final del episodio, al realizar la acción a_t estando el agente en el estado s_t. Se plantea a continuación, una deducción sencilla de algunas expresiones que nos permiten estimar la tabla Q. Al final del apartado, llegaremos a la expresión de la ecuación de Bellman.

Según se ha dicho, nos gustaría que $Q(s_t, a_t)$ guardara una estimación de las recompensas futuras que se obtendrían cuando el agente está s_t y se toma la acción a_t, es decir: $Q(s_t, a_t) = r_t + r_{t+1} + r_{t+2} + r_{t+3}$. Para ello, vamos a considerar que tenemos un entorno donde el número máximo de pasos es T (como ocurre en la mayoría de entornos de Gymnasium). Así pues:

$$Q(s_t, a_t) = \sum_{i=t}^{T} r_i = r_t + r_{t+1} + \ldots + r_T \tag{3}$$

Suponga que en el instante siguiente $t + 1$, el agente se ha movido hasta el estado s_{t+1}. El valor $Q(s_{t+1}, a_{t+1})$ de la matriz Q también debería almacenar la suma de recompensas hasta el final del episodio:

$$Q(s_{t+1}, a_{t+1}) = \sum_{i=t+1}^{T} r_i = r_{t+1} + r_{t+2} + \ldots + r_T \tag{4}$$

Podemos hacer:

$$Q(s_t, a_t) - Q(s_{t+1}, a_{t+1}) = r_t \tag{5}$$

Con lo que tenemos una primera forma de calcular un valor de la tabla Q:

$$Q(s_t, a_t) = r_t + Q(s_{t+1}, a_{t+1})$$

Analizamos, seguidamente, esta expresión. Durante la simulación, en un instante determinado, conocemos que el agente está en s_t y realiza la acción a_t, así pues, el elemento de la tabla a actualizar $Q(s_t, a_t)$ es fácil de identificar. Como consecuencia de la acción realizada, en ese mismo instante, el agente recibirá una recompensa r_t. El problema, ahora, consiste en sumar el término $Q(s_{t+1}, a_{t+1})$. De este término, conocemos s_{t+1}. Dado s_{t+1}, tendremos diferentes valores de $Q(s_{t+1}, a_{t+1})$, para cada una de las posibles acciones a_{t+1} que se pueden realizar en el instante siguiente. En consecuencia, ¿qué valor elegimos de $Q(s_{t+1}, a_{t+1})$? La solución consiste en elegir el máximo de estos valores, pues el agente elegirá la acción que maximice las recompensas futuras. Así, se propone una **primera forma para estimar la tabla Q**:

$$Q(s_t, a_t) = r_t + max_a(Q(s_{t+1}, a))$$

Nótese que esta ecuación se debe ejecutar de forma repetida para poder llegar a una estimación final, pues inicialmente se desconocen los valores de la tabla Q.

4.9. *Back-propagation*

Nos volvemos a fijar en la ecuación para actualizar Q:

$$Q(s_t, a_t) = r_t + max_a(Q(s_{t+1}, a))$$

Debemos fijarnos en que se produce una propagación hacia atrás en el tiempo (de $t+1$ a t). Una forma de entender el funcionamiento básico del aprendizaje es esta: **"imaginemos que un estado futuro s_{t+1} recibe una recompensa alta cuando consigue, por azar, la misión. Entonces el valor de $Q(s_t, a_t)$ se incrementará cuando a_t lleve al agente al estado s_{t+1}"**. De forma recurrente, podemos pensar que en otra iteración del algoritmo se aumentará el valor de $Q(s_{t-1}, a_{t-1})$ cuando a_{t-1} conduzca al agente a s_t. Vemos, por tanto, que debe existir una propagación hacia atrás desde los estados finales que están cerca de resolver el problema hasta los estados iniciales.

4.9.1. El factor de descuento γ

En algunos problemas, la suma de recompensas que se ha propuesto no resulta adecuada, pues la suma de recompensas pondera por igual los valores recibidos en el instante actual y los que se recibirán en el futuro (dentro de 100 pasos, por ejemplo). En la literatura es habitual modificar la suma de recompensas de la ecuación (3) por la siguiente expresión:

$$Q(s_t, a_t) = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \gamma^3 r_{t+3} + \ldots + \gamma^T r_T = \sum_{i=t}^{T} \gamma^{i-t} r_i \tag{6}$$

Con este factor $0 \leq \gamma \leq 1$ se consigue que el agente pueda dar menos peso a las recompensas más alejadas del instante t (los pasos siguientes). Igual que hicimos antes:

$$\begin{aligned} Q(s_t, a_t) &= r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \gamma^3 r_{t+3} + \gamma^4 r_{t+4} + \ldots \\ Q(s_{t+1}, a_{t+1}) &= r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \gamma^3 r_{t+4} + \gamma^4 r_{t+5} + \ldots \end{aligned}$$

Con esto, vemos que se puede escribir:

$$Q(s_t, a_t) = r_t + \gamma Q(s_{t+1}, a_{t+1})$$

E, igual que antes, obtenemos una estimación de la tabla Q:

$$Q(s_t, a_t) = r_t + \gamma \max_a(Q(s_{t+1}, a))$$

Obviamente, si ponemos $\gamma = 1$, tenemos el caso en el que el agente suma todas las recompensas sin ponderar. Si ponemos, por ejemplo, $\gamma = 0.5$, el agente considera más importantes las recompensas cerca del instante actual y será "miope" en relación con las recompensas futuras. Con esto, obtenemos otro parámetro γ que nos permitirá adaptar la técnica a la resolución de algunos problemas.

4.9.2. El factor de aprendizaje α

Siempre que planteamos un proceso de aprendizaje, deberíamos considerar una forma de actualización del valor actual de un parámetro. Nótese que en la última ecuación se hacía:

$$Q(s_t, a_t) = r_t + \gamma \max_a(Q(s_{t+1}, a_{t+1}))$$

Con lo cual, el valor de $Q(s_t, a_t)$ se sustituye directamente con el nuevo valor $Q(s_t, a_t) = r_t + \gamma \max_a(Q(s_{t+1}, a_{t+1}))$. En el ámbito de la inteligencia artificial, generalmente se propone utilizar un factor de aprendizaje α que permite añadir más o menos información a la estimación actual en cada paso del algoritmo. Imaginemos que en la iteración i tenemos una estimación $Q(s_t, a_t)_i$ de una celda de la tabla. La nueva estimación en la iteración siguiente es $Q(s_t, a_t)_{i+1}$. La nueva información de la que se dispone en el instante i es $\hat{Q}(s_t, a_t) = r_t + \gamma \max_a(Q(s_{t+1}, a_{t+1}))$. Se puede plantear una forma ponderada o filtrada de añadir información para la nueva estimación como:

$$Q(s_t, a_t)_{i+1} = (1 - \alpha) Q(s_t, a_t)_i + \alpha \hat{Q}(s_t, a_t) \tag{7}$$

Con este parámetro α (el factor de aprendizaje) podemos regular la forma en la que se añade nuevo aprendizaje a la tabla Q. Así pues:

- Si $\alpha = 1$ el aprendizaje es máximo, y el algoritmo olvida la información anterior en cada instante de simulación.
- Si $\alpha = 0$ el aprendizaje es nulo y no se añade nueva información a la tabla Q, lo que no es nada recomendable.

Esta expresión la podemos entender también como un filtro temporal que permite estimar $Q(s_t, a_t)$ usando una variable ruidosa.

4.9.3. La ecuación de Bellman

Con todo lo anterior, escribimos la ecuación de actualización completa sustituyendo en la ecuación (7) el valor de actualización propuesto $\hat{Q}(s_t, a_t) = r_t + \gamma \max_a(Q(s_{t+1}, a_{t+1}))$. Quedando:

$$Q(s_t, a_t)_{i+1} = Q(s_t, a_t)_i + \alpha[r_t + \gamma \max_a(Q(s_{t+1}, a)) - Q(s_t, a_t)_i] \tag{8}$$

Así pues, esta última ecuación se conoce como la **Ecuación de Bellman** (el matemático Richard Ernest Bellman la propuso en los años 50). La ecuación nos ofrece dos parámetros muy interesantes (α y γ) que podemos variar para adaptarnos a diferentes problemas de aprendizaje con Q-learning.

4.10. Resumen de parámetros

Así pues, en resumen, tenemos los siguientes parámetros, bien en la Ecuación de Bellman, o bien en la estrategia de generación de acciones durante el entrenamiento:

- α: *learning rate*. Regula cómo se actualizan los valores $Q(i, j)$ de la tabla con la nueva información que recibe el agente en cada instante. Si ponemos $\alpha = 1$ se sobreescribe la información anterior por completo con la nueva información. Por otra parte, si usamos $\alpha = 0$, el agente no "aprende", pues no utiliza la información nueva en modo alguno. Un valor de $\alpha = 0.5$ calcula en cada paso el valor medio entre el valor actual de $Q(i, j)$ y la nueva información aportada.
- γ: *discount rate* (o factor de descuento). Regula cómo se ponderan las recompensas futuras. Con $\gamma = 0$ el agente es "miope" y sólo busca maximizar la recompensa instantánea r_t, ignorando completamente las consecuencias a largo plazo. Con $\gamma \to 1$ (p. ej. 0.99), el agente es "visionario" y le da un peso muy similar a los beneficios cercanos y a los beneficios futuros: está dispuesto a tolerar pérdidas inmediatas si eso le asegura una gran recompensa más adelante.
- ϵ: es el parámetro que regula si el agente hará uso del conocimiento de la tabla Q o bien explorará nuevas soluciones aleatoriamente. El valor de ϵ se actualiza de cualquiera de las formas que se ha indicado antes. En los primeros episodios se busca que el agente pruebe nuevas soluciones. En los últimos episodios se busca que el agente "refine" y ajuste la solución.
- Número de episodios de entrenamiento: el número de episodios que usaremos para estimar Q.

5. Un primer entorno

Comenzaremos viendo un primer entorno sencillo en el que utilizaremos un algoritmo de Q-learning para que el agente aprenda a completar la misión con éxito. El entorno se observa en la figura 4. Se trata del entorno denominado *Cliff Walking*. En este entorno, el agente (un gnomo):

- Comienza siempre a moverse desde la posición [3, 0] de una rejilla de 4×12 y debe llegar al objetivo situado en la posición [3, 11].
- Si el agente llega a la posición deseada, el episodio finaliza.
- Hay un acantilado en todas las celdas [3, 1...10].
- Si el gnomo se mueve a una celda perteneciente al acantilado, volverá a la casilla de salida.

El objetivo consiste en que el agente sea capaz de llegar desde la casilla de salida hasta el destino en el menor número de pasos de simulación posible. Este entorno es, en extremo sencillo. Fácilmente, podríamos escribir a mano la tabla Q óptima que resolvería el problema, haciendo que el gnomo siguiera la secuencia obvia: Arriba, derecha, derecha, derecha, ..., abajo. No obstante, se desea comenzar con este entorno sencillo para experimentar con la técnica de Q-learning. En apartados posteriores se resolverán problemas más complejos. De esta manera, se busca que el estudiantado:

- Se familiarice con el algoritmo Q-learning y lo programe en python.
- Programe la regla de actualización de la ecuación de Bellman.
- Juegue con diferentes valores de los hiperparámetros del algoritmo y observe el resultado.

Puedes consultar toda la documentación sobre este entorno aquí: https://gymnasium.farama.org/environments/toy_text/cliff_walking/

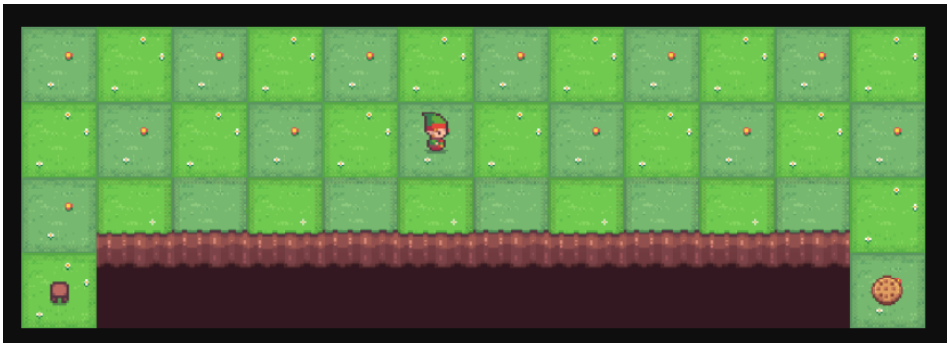


Figura 4: El entorno Cliff Walking

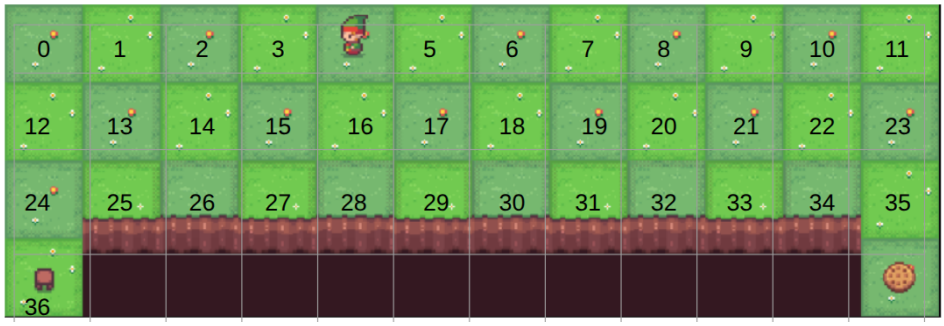


Figura 5: La codificación de estados del problema.

5.1. Estados, acciones y recompensas

El entorno de Gymnasium contiene $4 \times 12 = 48$ posibles estados. De entre estos estados, solamente son posibles $3 \times 12 + 1 = 37$ estados, pues el agente puede estar en cualquiera de las celdas de las 3 filas superiores y, además, puede encontrarse en la casilla de salida. El agente no puede estar en ninguna de las celdas del acantilado ni en la celda de meta, pues esto ocasiona el final del episodio.

5.1.1. Codificación del estado

El estado del problema debe codificar la posición del gnomo en cada instante. Una solución obvia consistiría en definir el estado con las coordenadas (i, j) de cada celda, recordando que en python ambas se numeran comenzado en 0. Sin embargo, resulta más conveniente codificar los estados con un único entero según se muestra en la figura 5. Así pues, cuando el agente se encuentra en la posición (i, j) del entorno, el estado entero se codifica como $s = i \times n_c + j$, donde n_c es el número de columnas del entorno (12). Por ejemplo, la posición de salida del agente se codifica con el entero 36 ($36 = 3 \times 12 + 0$).

5.1.2. Acciones

En cada posición el agente puede realizar las siguientes acciones, codificada cada una de ellas con un número entero:

- 0: arriba.
- 1: derecha.
- 2: abajo.
- 3: izquierda

5.1.3. Recompensas

El gnomo recibe una recompensa de -1 en cada paso del algoritmo. Se obtiene una recompensa de -100 cuando el agente se cae por el acantilado. Se desea que el gnomo alcance la meta con una suma de recompensas en el episodio máxima.

5.1.4. La tabla Q

Con todo lo anterior, la tabla Q tiene la forma de esta matriz:

$$Q = \begin{bmatrix} q_{0,0} & q_{0,1} & q_{0,2} & q_{0,3} \\ q_{1,0} & q_{1,1} & q_{1,2} & q_{1,3} \\ \dots & \dots & \dots & \dots \\ q_{36,0} & q_{36,1} & q_{36,2} & q_{36,3} \end{bmatrix}$$

La matriz tiene tantas filas como estados posibles se han codificado (48) y tantas columnas como posibles acciones a realizar en cada estado (4). Cada uno de los elementos $q_{i,j}$ almacena el valor de $Q(i, j)$ que estima las recompensas futuras que se obtendrían al realizar la acción j cuando el agente se encuentra en el estado i. La técnica Q-learning propone una manera de estimar esta matriz para maximizar la recompensa de las acciones realizadas por el agente. En resumen: debemos calcular cada elemento de la matriz `q_table` para que `q_table[s][a]` estime la recompensa futura (sobre todos los estados futuros hasta el final del episodio). La tabla `q_table` la podemos encontrar si entramos en la clase `QLearning` (directorio `practice_qlearning_1/qlearning`):

```
class QLearning():
def __init__(self, environment, params):
    self.env = environment
    self.state_size = self.env.observation_space.n
    self.action_size = self.env.action_space.n
    self.q_table = np.zeros((self.state_size, self.action_size))
```

6. ACTIVIDADES I

6.1. Programación de un algoritmo de Q-learning

Ejercicio 6.1

Si abrimos el script `cliff_q_learning_test.py`, encontraremos el siguiente script. Léalo con atención.

```
from qlearning.qlearning import QLearning
import gymnasium as gym

def test_qlearning():
    environment = gym.make('CliffWalking-v1', render_mode="human")
    qlearning = QLearning(environment=environment)
    qlearning.create_random_q_table()
    #qlearning.read_q_table(filename='qtable.npy')
    print('Testing mode!')
    qlearning.test(total_episodes_test=1)

if __name__ == "__main__":
    test_qlearning()
```

La clase `QLearning` se proporciona programada **parcialmente**, para que el estudiante la complete. Importante:

- En estos momentos la matriz Q no se conoce, con lo que el script genera una matriz Q aleatoria con el método `create_random_q_table`.
- Si se utiliza la línea `qlearning.read_q_table(filename='qtable_cliff.npy')` se cargará la tabla Q desde fichero. **Esto se deberá hacer cuando ya se haya entrenado y tengamos este fichero (script de train).**
- Si se ejecuta el script, el agente intentará resolver la tarea utilizando una matriz Q aleatoria: realizará movimientos de forma aleatoria sin llegar a su objetivo.

EJERCICIO

- Ejecute el script `cliff_q_learning_test.py`, al menos, 15 veces. Observe el comportamiento del agente. ¿Se consigue completar la tarea?
- ¿Cuál es la recompensa total de un episodio en el que el gnomo completa la tarea de forma óptima?

El Teorema del mono infinito afirma que un mono pulsando teclas al azar sobre un teclado durante un periodo de tiempo indefinido podrá escribir finalmente "Hamlet" de William Shakespeare (https://es.wikipedia.org/wiki/Teorema_del_mono_infinito). De la misma manera, podríamos generar tablas Q aleatorias y pensar que alguna de ellas permitiría que el gnomo fuera desde la casilla de salida hasta la casilla de llegada. Si, por algún casual, ocurre esto (la probabilidad de que ocurra la trayectoria óptima es bastante fácil de calcular), este caso sería fácil de detectar y nos permitiría aproximar la tabla Q (pues cada valor de Q(i,j) tendría el valor total de recompensa del episodio). Sin embargo, la tabla Q calculada así dará difícilmente una solución óptima del problema: con mucha suerte el gnomo llegará a la casilla final... y con muchísima suerte la trayectoria realizada será óptima. También podríamos pensar qué hacer con diferentes tablas Q que resuelvan el problema de forma subóptima... para, de alguna manera intentar optimizarlo. La idea que se propone en esta práctica es diferente de la "del mono". Así pues, utilizaremos la ecuación de Bellman para actualizar los valores de Q en cada instante de simulación.

Ejercicio 6.2

EJERCICIO
Abra el fichero `practice_qlearning_1/qlearning/qlearning.py`. Busque la clase `QLearning()` y halle el método de test:

```
def test(self, total_episodes):
```

Examine el método test de la clase `QLearning`, y entienda su funcionamiento. **Ayuda**:

- `state, info = self.env.reset()`: resetea el entorno a su estado inicial.
- `action = np.argmax(self.q_table[state])`: la tabla se guarda en el array `self.q_table`. Esta matriz tiene 48 filas y 4 columnas. Cada fila codifica un estado y cada columna una de las 4 acciones. **El valor máximo de cada fila** le indica al agente la acción más beneficiosa para maximizar las recompensas futuras. Así pues, la acción se calcula como el índice que corresponde con el máximo de los valores de la fila del estado actual, en python: `action = np.argmax(self.q_table[state])`.
- Para ejecutar la acción sobre el entorno, se usa la función `step(action)`: `next_state, reward, terminated, truncated, info = self.env.step(action)`

Ejercicio 6.3

EJERCICIO
Abra el script `cliff_q_learning_train.py`. Verá que el script es similar al script de *test* vimos antes, con la diferencia de que en este caso guardamos la tabla Q al finalizar el método *train*. La idea es la habitual en *Machine Learning*: a) Entrenar y guardar Q. b) Testear: cargar Q y usarla para que el agente opere sobre el entorno. Se presenta el código, a continuación:

```
def train_qlearning():
    environment = gym.make('CliffWalking-v1', render_mode='human')
    qlearning = QLearning(environment=environment)
    print('LET US LEARN NOW!')
    qlearning.train(total_episodes_train=300)
    qlearning.save_q_table(filename='qtable.npy')

if __name__ == "__main__":
    train_qlearning()
```

Use `render_mode=human` si quiere ver la evolución del agente en el entorno. Normalmente, no haremos esto, pues ralentiza el aprendizaje:

```
environment = gym.make('CliffWalking-v1', render_mode='human')
```

Así pues, durante el aprendizaje, usaremos la siguiente configuración:

```
environment = gym.make('CliffWalking-v1')
```

Si entramos en el método train (clase Qlearning), veremos esto:

```
def train(self, total_episodes_train):
   print("Training started!\n\n")

   raise NotImplementedError('EJERCICIO 6.3 sin completar':
   debe completar QLearning.train()
   return self.q_table
```

Complete el método `train` de la clase `QLearning`. **Nota**: deje el método `update_q_table` vacío (se completará en el apartado siguiente). Como ayuda se proporciona el pseudo código que permite entrenar y calcular la tabla Q. La mayoría de las funciones ya se han visto en el método de test anterior.

```
def train(self, total_episodes):
    epsilon_greedy = EpsilonGreedyGeom(...)
    - Repita durante un total de  total_episodes:
        - Resetee el entorno.
        while True:

            - Tome una decisión de exploración o explotación:
            (epsilon_greedy.random_action() devuelve True si hay que hacer una acción
            aleatoria).

            - Aplique la acción sobre el entorno.
            next_state, reward, terminated, truncated, info = self.env.step(action)

            - Ejecute cada episodio hasta que terminated o truncated sea True.
            - Actualice la tabla Q con la ecuación de Bellman. Esto se programa en el
            método:
            self.update_q_table(state, action, next_state, reward)

            - Se actualiza la tabla Q con la nueva información:
            update_q_table(state, next_state, action, reward)

            - Pasamos al nuevo estado
            state = next_state
        # reducimos epsilon en cada episodio
        epsilon_greedy.step()
```

Este pseudocódigo se puede resumir rápidamente: se simulan `total_episodes` de entrenamiento. En cada episodio, se resetea el entorno a un estado inicial y se ejecutan una serie de pasos de simulación. El valor de ϵ y la forma en la que se reduce están definidos por la clase `EpsilonGreedyGeom`. Esta clase tiene una función `random_action` que con probabilidad ϵ devuelve `True` (y se genera una acción aleatoria) y con probabilidad $1-\epsilon$ devuelve `False` y se obtiene la acción más adecuada de la tabla Q (según el valor de $Q(s,a)$ estimado hasta el momento). La clase `EpsilonGreedyGeom` se encarga de mantener el valor de epsilon y de actualizarlo (método `.step()`). Cada acción se aplica sobre el agente y el entorno (`apply_action`), lo que realiza la simulación instantánea del entorno y devuelve un nuevo estado y recompensa. Finalmente, se actualiza la tabla Q (ejercicio siguiente) con la función `update_q_table`.

Ejercicio 6.4

EJERCICIO
Complete la función `update_q_table` usando la expresión de la Ecuación de Bellman:

$$Q(s_t,a_t)_{i+1} = Q(s_t,a_t)_i + \alpha[r_t + \gamma \max_a(Q(s_{t+1},a)) - Q(s_t,a_t)_i] \tag{9}$$

Se da aquí el prototipo:

```
def update_q_table(self, state, action, next_state, reward):
    self.q_table[state, action] = ...
```

Importante: se ha definido una variable de la clase para almacenar la tabla Q: `self.q_table`. También se han definido las variables de la clase `self.learning_rate` (α) y `self.discount` (γ).

6.1.1. El ciclo de *train-test*

En estos momentos ya contamos con un algoritmo funcional para el aprendizaje. Una aplicación de aprendizaje automático se resume en los siguientes pasos:

a) Ejecutamos un entrenamiento durante un número de episodios: el código del script `cliff_q_learning_train.py`. En este entrenamiento, usaremos un conjunto de hiperparámetros: α, γ, N y la estrategia de ϵ.

b) Evaluamos el resultado usando un script de test: `cliff_q_learning_test.py`

c) Si estamos satisfechos con el resultado del test $\Longrightarrow$ hemos terminado. Si estamos insatisfechos $\Longrightarrow$ `GOTO a)`.

Debemos darnos cuenta de:

- Durante el entrenamiento se estima la matriz Q. Al final del entrenamiento esta matriz Q se guardará en el fichero `qtable.npy`.

```
qlearning.save_q_table(filename='qtable.npy')
```

- Realizado el entrenamiento, se deberá probar la tabla Q para evaluar su conocimiento sobre la misión. El script de test deberá cargar la matriz Q desde el fichero `qtable.npy`. Así pues, se deberá modificar el script de test deberá hacer:

```
qlearning.read_q_table(filename='qtable.npy')
```

α	γ	"Ecuación de actualización"	¿Aprende?
0	0	$Q(s_t, a_t)_{i+1} = Q(s_t, a_t)_i$	¿?
0	1	$Q(s_t, a_t)_{i+1} = Q(s_t, a_t)_i$	¿?
1	0	¿?	¿?
1	1	¿?	¿?
0.5	0.99	¿?	¿?

Cuadro 1: Valores de α y γ para el ejercicio 6.1.1.

Ejercicio 6.5

EJERCICIO
Ejecute el algoritmo de entrenamiento durante 100 episodios (`cliff_q_learning_train.py`). Use los parámetros: $\epsilon_0 = 1$, $N_{target} = 0.25$ y $\epsilon_{min} = 0.01$. Después:

a) Cambie los parámetros α y γ de acuerdo con la tabla 1. Puede hacer:

```
environment = gym.make('CliffWalking-v1', render_mode="human")
params = {'alpha': 1.0, 'gamma': 0.99, 'percentage_target': 0.25}
qlearning = QLearning(environment=environment, params=params)
```

b) Para cada conjunto de parámetros, ejecute el script de *train* y, a continuación, ejecute el script de *test*. Observe el resultado.

c) Complete los valores de la tabla 1: rellene la columna denominada "ecuación" y conteste sí/no, observando si el algoritmo aprende a resolver la tarea (evalúe el método de test y observe si el gnomo completa la misión).

Según se ha podido comprobar, la función que permite actualizar los valores de la tabla Q depende de los valores de α y γ. Así pues, la ecuación de Bellman es una "navaja suiza" que se puede adaptar a la resolución de múltiples problemas de aprendizaje al variar α y γ. Sin embargo, es importante conocer cómo funcionan estos parámetros, pues hemos visto que es fácil caer en casos que no permiten obtener un aprendizaje eficaz. En consecuencia, es importante entender qué ocurre cuando se modifican los parámetros α y γ de la ecuación de Bellman. En resumen, se ha comprobado que con $\alpha = 0$ no se produce ningún aprendizaje, obviamente y que, también, la estrategia $Q(s_t, a_t) = r_t$ no produce, tampoco, ningún conocimiento (pues no propaga la información de otros estados ulteriores). Esta estrategia $Q(s_t, a_t) = r_t$ es **inadecuada**, pues está almacenando solamente la recompensa actual y no la suma de recompensas hasta el final.

7. Resumen

El código siguiente permite resolver un problema de aprendizaje e incluye todos los conceptos que hemos visto hasta ahora:

```
def train(self, total_episodes):
    self.results = []
    print('Training started!', flush=True)
    epsilon_greedy = EpsilonGreedyGeom(epsilon_max=self.epsilon_max,
                                       epsilon_min=self.epsilon_min,
                                       total_episodes=total_episodes,
                                       percentage_target=self.percentage_target)
    recent_rewards = []
    for episode in range(total_episodes):
        state, info = self.env.reset()
        total_reward = 0
```

```
            while True:
                if epsilon_greedy.random_action():
                    action = self.env.action_space.sample()  # Explore
                else:
                    action = np.argmax(self.q_table[state])  # Exploit
                next_state, reward, terminated, truncated, info = self.env.step(action)
                total_reward += reward
                self.update_q_table(state, action, next_state, reward)
                if terminated or truncated:
                    break
                state = next_state
            epsilon_greedy.step()
        print("Training finished! Your Q-table is optimized.")
        return self.q_table

    def update_q_table(self, state, action, next_state, reward):
        # Update Q-table using the Bellman Equation
         self.q_table[state, action] = (self.q_table[state, action] +
                    self.learning_rate * (reward + self.discount_rate *
                    np.max(self.q_table[next_state]) -
                    self.q_table[state, action]))

    def test(self, total_episodes, save_results=False):
        recent_rewards = []
        for episode in range(total_episodes):
            state, info = self.env.reset()
            total_reward = 0
            while True:
                action = np.argmax(self.q_table[state])
                next_state, reward, terminated, truncated, info = self.env.step(action)
                total_reward += reward
                if self.env.render_mode == 'human':
                    time.sleep(.1)
                if terminated or truncated:
                    break
                state = next_state
            recent_rewards.append(total_reward)
            if save_results:
                self.results.append([episode, total_reward, 0])
            print(f"Episode {episode:4d} | Total reward: {total_reward:6.1f}", end='\r')
            if episode % 10 == 0:
                print()
        return recent_rewards
```

8. Otro entorno más desafiante: Taxi v3

En este entorno existe un taxi que puede moverse por el mapa que simula una ciudad (figura 6). La misión consiste en recoger a un pasajero de una posición determinada para dejarlo en el hotel. Tanto la posición de recogida como la de destino pueden variar en cada simulación.

En este problema, a diferencia del entorno *Cliff Walking*, los estados iniciales serán diferentes. Esto incluye variaciones de la posición inicial del taxi, de la posición inicial del pasajero y del destino. Se excluyen de estos estados iniciales (la librería gymnasium los elimina automáticamente) los casos ilegales en los que el pasajero ya se encuentra dentro del vehículo, o si el vehículo y el pasajero se encuentran ya en el destino.

8.1. Descripción del entorno

8.1.1. Acciones

El taxi puede realizar las siguientes acciones:

- 0: Move south (down).
- 1: Move north (up).
- 2: Move east (right).
- 3: Move west (left).
- 4: recoger al pasajero.
- 5: dejar al pasajero.

8.1.2. Estado

El estado de este problema debe codificar:

- La posición del taxi en una rejilla de 5×5.
- La posición del pasajero:
  - 0: Red
  - 1: Green
  - 2: Yellow
  - 3: Blue
  - 4: In taxi
- Los posibles destinos del pasajero (la posición del hotel):
  - 0: Red
  - 1: Green
  - 2: Yellow
  - 3: Blue

Con todo esto, hay 500 estados posibles, pues existen:

- 25 posiciones posibles en las que se encuentra el taxi.
- Hay 5 posiciones posibles para el pasajero (4 cuadros de colores y, además, el caso en el que el pasajero está montado en el coche).
- Finalmente, hay 4 destinos posibles para el pasajero.

En total son $n_s = 25 \times 5 \times 4$ estados posibles. El estado se codifica como un entero que se calcula como:
`state=((taxi_row * 5 + taxi_col) * 5 + passenger_location) * 4 + destination`

No obstante, de entre todos los estados anteriores, solamente hay 404 que pueden ser alcanzados en un episodio. Los estados que faltan corresponden con situaciones en las que el pasajero se encuentra ya en la misma posición de destino (pues en ese momento, el episodio acaba). El caso en el que el taxi y el pasajero están en la misma casilla de destino puede observarse justo después de que el episodio acabe.

8.1.3. Estado inicial

El estado inicial se genera de forma aleatoria de entre los estados válidos donde el pasajero no se encuentra ya en el destino ni dentro del taxi. En total, existen 300 estados iniciales diferentes.

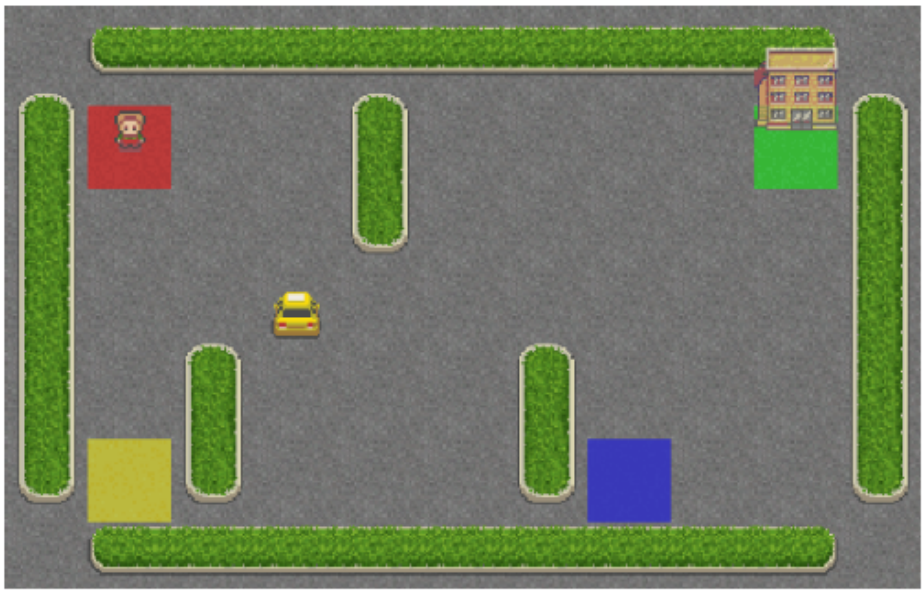


Figura 6: El entorno Taxi.

8.1.4. Recompensas

En el caso del taxi, las recompensas se definen así:

- -1 por cada paso, si no se obtiene otra recompensa.
- +20 por dejar al pasajero en el destino (correcto).
- -10 por recoger o dejar al pasajero de forma "ilegal".

Así pues, el hecho de realizar un paso (step) conlleva una recompensa negativa, pues buscamos que el taxi realice el transporte del pasajero de inicio a final en el mínimo tiempo posible (se desea maximizar la suma de recompensas recibidas). Cuando se deja al pasajero, se obtiene una recompensa positiva (+20). Nótese que se obtiene una recompensa negativa por acciones ilegales de recogida y reparto de la persona: pues podría ocurrir que el agente decidiera dejar al pasajero lejos de su destino o bien intentar recogerlo en una casilla donde no estuviera la persona).

Un acción que conlleve que el Taxi no se mueva, como, por ejemplo, intentar ir contra un obstáculo, incurre en una recompensa negativa (-1).

8.1.5. Final del episodio

El episodio acaba cuando el taxi deja al pasajero en la posición adecuada (`terminated=True`). El episodio acaba cuando la duración del episodio excede los 200 pasos (`truncated=True`).

8.2. ACTIVIDADES II

Los ficheros de esta parte de la práctica son:

- Entrenamiento: `taxi_q_learning_train.py`.
- Test: `taxi_q_learning_test.py`.

Si prestamos un segundo de atención a la clase QLearning que habéis programado, vemos lo siguiente:

```
class QLearning():
def __init__(self, environment, params):
    self.env = environment
```

```
self.state_size = self.env.observation_space.n
self.action_size = self.env.action_space.n
self.q_table = np.zeros((self.state_size, self.action_size))
self.learning_rate = params.get('alpha', 1.0)  # alpha
self.discount_rate = params.get('alpha', 0.9)  # gamma
self.percentage_target = 0.25
self.max_epsilon = params.get('max_epsilon', 1.0)
self.min_epsilon = params.get('min_epsilon', 0.01)
```

En efecto, cuando creamos un objeto de la clase `QLearning`, la tabla Q se inicializa automáticamente consultando el tamaño del estado y de las acciones con el entorno que le pasamos (`environment`). Así pues, el mismo código que hemos usado para resolver el entorno *Cliff Walking* lo podemos usar para el problema del Taxi y para cualquier otro problema de Q-learning en el que podamos describir el estado de forma discreta. En la parte final del método `__init__` se guardan los parámetros que hemos comentado anteriormente: α, γ y la estrategia ϵ-greedy que definen los parámetros de la Ecuación de Bellman y la forma de hacer la exploración/explotación de Q durante el entrenamiento. Para poder modificar estos parámetros, en la clase habrás visto que se hace `self.min_epsilon = params.get('min_epsilon', 0.01)`: esta parte del código busca un parámetro en el diccionario y, si no se encuentra, guarda un valor por defecto.

En esencia, podemos entrenar el entorno del taxi directamente y solamente nos debemos preocupar de seleccionar unos parámetros α, γ, ϵ_{max}, ϵ_{min} y N_{target} (`self.percentage_target`) que permitan optimizar el aprendizaje del agente. En el apartado siguiente experimentaremos con diferentes valores de estos parámetros y observaremos los resultados.

8.3. ¿Está aprendiendo el agente?

Comenzamos ejecutando directamente el script de entrenamiento unos hiperparámetros para los que pensamos que funcionará el aprendizaje.

Ejercicio 8.1

EJERCICIO
Ejecute el script de entrenamiento `taxi_q_learning_train.py` con 300 episodios de entrenamiento y los siguientes valores de *learning rate* y *discount rate*: $\alpha = 0.1$, $\gamma = 0.5$.

```
def train_qlearning():
    environment = gym.make('Taxi-v3')
    params = {'alpha': 1.0, 'gamma': 1.0}
    qlearning = QLearning(environment=environment, params=params)
    qlearning.train(total_episodes=300)
    qlearning.save_q_table(filename='qtable_taxi.npy')

if __name__ == "__main__":
    train_qlearning()
```

Ejecute, después, el script de test `taxi_q_learning_test.py`. ¿Se consigue completar la misión? ¿Ha aprendido el agente?

Vemos que el problema del taxi, es más complejo que el problema del acantilado, pues hay un mayor número de estados y acciones posibles. En consecuencia, previsiblemente, se necesitará un mayor número de episodios de entrenamiento para que se produzca un aprendizaje significativo. En el ejercicio anterior (salvo que hayamos sido muy afortunados) no se consigue resolver la tarea correctamente. Como no se ha producido un buen aprendizaje, nos preguntamos:

- ¿Debemos aumentar el número de episodios para el entrenamiento?
- ¿O debemos cambiar los valores de los parámetros γ y α?
- ¿Debemos cambiar la estrategia ϵ-greedy?

Efectivamente, es necesario saber qué esta pasando en cada episodio durante el entrenamiento. En caso contrario, iremos "a ciegas". Cuando nos enfrentamos a problemas de aprendizaje más complejos, **es vital que sepamos si efectivamente se está produciendo algún tipo de aprendizaje, pues durante el entrenamiento se está utilizando tiempo de cómputo y energía (ambos valiosos)**. En caso de que sospechemos que no se esté obteniendo un buen aprendizaje, deberíamos parar el entrenamiento y volver a la fase de diseño. Hay diferentes formas de evaluar el aprendizaje durante la fase de entrenamiento. **Una forma sencilla es la siguiente**: en un porcentaje de episodios de entrenamiento (por ejemplo, en uno de cada 10), se desactiva la opción ϵ-greedy y se toman todas las acciones del episodio en base a la tabla Q (se evitan las acciones aleatorias). Mientras se hace eso, se calcula la recompensa total del episodio.

Ejercicio 8.2

EJERCICIO
Se desea medir el aprendizaje durante los diferentes episodios de entrenamiento. Para ello, es necesario modificar el método de entrenamiento (según se ha explicado anteriormente). Partiendo del script `taxi_q_learning_train.py`:

- Modifique el método de train de la clase `QLearning`.
- Cree una método denominado `self.inline_test()` en la clase `QLearning`.
- En el método de train, cada 10 episodios de entrenamiento el método `self.inline_test()` deberá ejecutar 10 episodios de test. La función debe devolver **la recompensa media de esos 10 episodios**, pues el resultado de cada test puede ser diferente.
- En cada test *inline*, solamente se explota el conocimiento inherente a Q (no se toman acciones aleatorias).
- En el método train se debe guardar el resultado de cada llamada a `self.inline_test()`. Se debe devolver la lista con el resultado de cada test y calcular el resultado medio.
- Plotee los resultados al finalizar el entrenamiento.

Haga:

- Un entrenamiento con $\alpha = 0.9$ y $\gamma = 0.1$ de 1500 episodios (habrá 150 episodios de test).
- Un entrenamiento con $\alpha = 0.5$ y $\gamma = 0.5$ de 1500 episodios (habrá 150 episodios de test).
- Las combinaciones cruzadas de parámetros que faltan.

```
def train_qlearning():
    environment = gym.make('Taxi-v4')
    params = {'alpha': 0.5, 'gamma': 1.0}
    qlearning = QLearning(environment=environment, params=params)
    results = qlearning.train(total_episodes=1500)
    qlearning.save_q_table(filename='qtable_taxi.npy')
    plt.figure()
    plt.plot(results)
    plt.ylabel('Mean reward')
    plt.xlabel('Episodes (inline tests)')
    plt.show()
```

Se muestra en la figura 7 un resultado típico de un entrenamiento de 2000 episodios con $\alpha = 0.5$ y $\gamma = 1.0$, usando una función de ϵ geométrica 25/75. Vemos cómo aumenta la recompensa media (medido con un *inline test*). Hacia los episodios finales la recompensa media se estabiliza, acabando en valores positivos que resuelven el problema del entorno Taxi. Un resultado peor se obtiene con $\alpha = 0.1$ y $\gamma = 1.0$ (figura 8).

Hecha la prueba anterior, posiblemente, no tengamos muy claro cuáles son los parámetros que per-

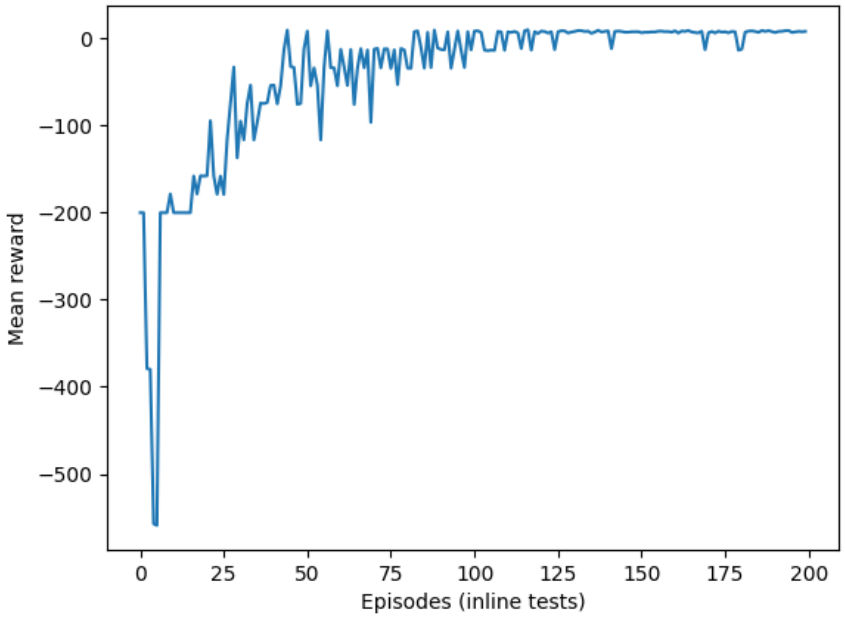


Figura 7: Recompensa media durante el entrenamiento con $\alpha = 0.5$ y $\gamma = 1.0$.

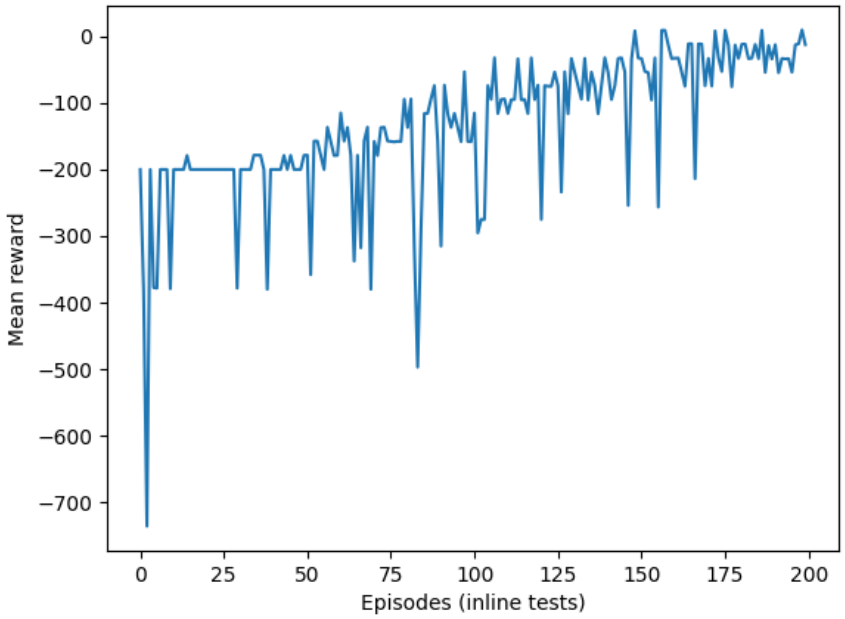


Figura 8: Recompensa media durante el entrenamiento con $\alpha = 0.1$ y $\gamma = 1.0$.

miten obtener un mejor aprendizaje del entorno. En general, los valores óptimos de los hiperparámetros dependen fuertemente del entorno. Una forma de hallar los parámetros óptimos para un entorno es realizar varios entrenamientos diferentes del entorno con todas las combinaciones de α y γ posibles.

Ejercicio 8.3

EJERCICIO
Abra el script `taxi_q_learning_train_opt.py`. Observe que aquí se plantea algo diferente:

- Se realiza un entrenamiento con diferentes valores de α y γ.
- Se testea el resultado durante un buen número de episodios y se guarda el resultado medio en un array bidimensional.

Realice lo siguiente:

- Utilice `np.linspace(0.05, 1, 5)` para hallar 5 valores de α y γ en el intervalo $[0.05, 1]$.
- Modifique `total_epidodes_train = 1500, total_epidodes_test = 500`.
- Observe el mapa de calor resultante.

Nota: se está haciendo un barrido con "fuerza bruta" de todos los posibles valores de parámetros, con lo que es normal que el script tarde bastante en ejecutarse.
Nota: se han probado 25 combinaciones diferentes de parámetros. Si quisiéramos "afinar" más el resultado, podríamos hacer, por ejemplo `np.linspace(0.5, 1, 15)` en ambos parámetros, pues ya hemos podido descartar algunas combinaciones que no producen buenos resultados.

Deberías obtener un resultado como el de la figura 9. Se muestra un mapa de calor donde tenemos varios puntos de α y γ con valores de aprendizaje similares. Deberíamos aumentar el número de episodios de *train* y *test* para poder encontrar diferencias significativas (incrementando enormemente el tiempo de cómputo total). En base a los resultados, vemos que podemos obtener buenos valores de aprendizaje con $\alpha \in [0.4, 0.9]$ y $\gamma = 1.0$.

9. Instalación

Los equipos de prácticas tienen todo lo necesario para realizar la práctica. En este apartado se dan algunas instrucciones para que puedas instalar todo lo necesario en otro portátil/PC y poder trabajar de forma autónoma en tu propio PC/portátil. Las instrucciones se ofrecen para un sistema Linux/Ubuntu. Comenzamos instalando el programa git:

```
$ sudo apt install git
```

Comenzamos por descargarnos el repositorio de prácticas. Es un repositorio que se encuentra en Github:

```
$ cd /home/usuario/Desktop
$ git clone https://github.com/4rtur1t0/Intro2AI
```

Veremos que tenemos un directorio llamado Intro2AI en el escritorio. Seguimos instalando algunos paquetes necesarios para la creación de entornos virtuales de python `virtualenv`:

```
sudo apt install python3-pip python3-venv python3-dev -y
```

Seguidamente, creamos un entorno virtual:

```
$ cd /home/usuario
$ mkdir Applications
$ cd Applications
$ virtualenv venvAI
$ source /home/usuario/Applications/venvAI/bin/activate
```

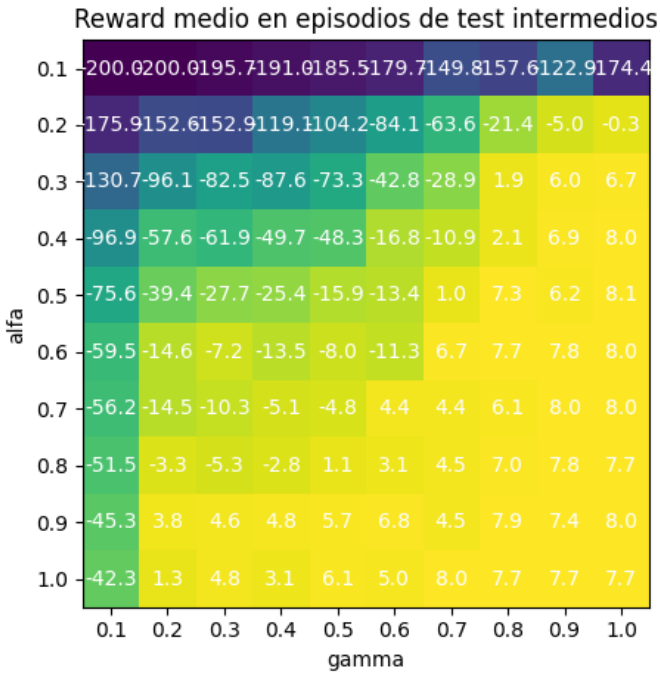


Figura 9: Aprendizaje medio de diferentes combinaciones de α y γ en episodios de test intermedios. Entorno: Taxi.

Con esto hemos creado un entorno virtual de python en el directorio `/home/usuario/Applications/venvAI` y lo hemos activado. Instalamos ahora algunos paquetes para este entorno de python:

```
$ cd /home/usuario/Applications/venvAI/bin
$ sudo apt install swig
$ ./pip install -r /home/arturo/Desktop/Intro2AI/requirements.txt
$ ./pip install stable-baselines3 swig torch
$ ./pip install tensorboard
$ ./pip install "gymnasium[atari,accept-rom-license]" stable-baselines3 shimmy
```

Cuando usemos un editor de python, como, pyCharm o Visual Studio, necesitaremos asociar el entorno virtual que hemos creado con el proyecto.

Referencias

[SB18] Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. The MIT Press, Cambridge, Massachusetts, 2nd edition, 2018.